LSA-PAST: Layer-Specific Adaptation and Prototype Transport for Compositional Incremental Learning

Kyle Stein[1] Andrew Arash Mahyari[2] Guillermo Francia III[1] Eman El-Sheikh[1] Jiangpeng He[3]
[1]University of West Florida, Pensacola, FL, USA
[2]Florida Institute for Human and Machine Cognition, Pensacola, FL, USA
[3]Indiana Univeristy, Bloomington, IN, USA
{kstein,gfranciaiii,eelsheikh}@uwf.edu, amahyari@ihmc.org, jhe2@iu.edu

Abstract

*Class-Incremental Learning (CIL) enables models to recognize new classes over time while retaining prior knowledge, but many real-world problems are compositional, where visual concepts arise from combinations of primitives such as states (i.e., red, shiny) and objects (i.e., shoe, car). Standard CIL methods treat each composition as an independent class, ignoring shared primitives and providing no mechanism to stabilize their representations as new compositions arrive. This causes parameter updates to entangle primitives and induce drift that degrades performance on earlier compositions. Compositional Incremental Learning (composition-IL) formalizes this setting by introducing new state–object compositions across tasks. In this paper, we propose Layer-Specific Adaptation with Prototype-Anchored Adapter-Space Transport (**LSA-PAST**), a parameter-efficient method tailored to composition-IL. LSA integrates Low-Rank Adaptation (LoRA) into a frozen ViT with depth-specialized adapters for states, objects, and compositions. PAST then maps past, anchored prototypes into the current adapter space through a rehearsal-free affine transform fit on current-task data, enabling old prototypes to remain compatible with new adapters. Extensive experiments demonstrate that LSA-PAST achieves state-of-the-art (SOA) accuracy-forgetting trade-off performance for composition-IL.*

1. Introduction

Deep learning has achieved remarkable success across visual recognition tasks, from fine-grained identification to large-scale categorization [2, 16, 26]. This success typically rests on the brittle assumption that the world can be captured in a single, static training set. However, in real-world deployments, data arrives sequentially and models must incorporate new knowledge without full retraining or access to past data. Class-Incremental Learning (CIL) [20, 27, 29, 32, 39] formalizes this setting where a learner is exposed to a sequence of tasks with expanding label space and must acquire new classes while preserving previously learned knowledge, reinforcing the issue of catastrophic forgetting [8]. Recent CIL methods adopt parameter-efficient fine-tuning (PEFT) of pre-trained models to alleviate forgetting through prompt-tuning [40, 44, 45] and adapters [15, 25, 55]. However, this progress is largely measured in object-only settings where classes are treated as singular entities, an assumption misaligned with the compositional nature of real-world concepts.

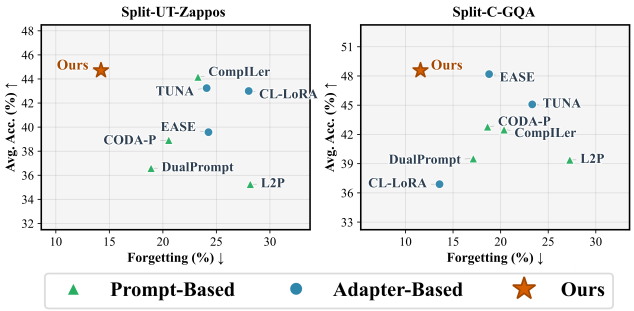


Figure 1. Accuracy–forgetting trade-off on composition-IL benchmarks for adapter- and prompt-based methods.

Concepts in the visual world are often *compositional*, combining primitives such as states (e.g., *striped*, *leather*) and objects (e.g., *shirt*, *shoe*) [7, 21, 22]. This structure is central to human cognition, yet most CIL methods and benchmarks treat classes as atomic and ignore compositionality. Compositional Incremental Learning (composition-IL), introduced in [52], addresses this gap by learning to acquire new state–object pairs over tasks while retaining performance on prior compositions. A key difficulty in composition-IL is *state confusion*: models often recognize the object but confuse its state (e.g., "shoe" vs. "leather shoe" or "canvas shoe"). Pretrained models bias representations toward object identity [10, 35], causing feature entanglement among compositions sharing an object [13, 30]. Furthermore, standard pretraining often lacks the

fine-grained sensitivity to texture and material required for state discrimination [10, 46]. These issues worsen when object-centric CIL methods are applied to compositions. Fig. 1 shows that prior, object-centric CIL methods yield poor accuracy-forgetting trade-offs, while our method provides the strongest overall balance across all benchmarks.

In this work, we propose Layer-Specific Adaptation with Prototype-Anchored Adapter-Space Transport (LSA-PAST) for composition-IL. Our approach introduces *Layer-Specific Adaptation* (LSA) with three depth-specialized Low-Rank Adaptation (LoRA) [18] adapters that exploit the hierarchical processing structure of vision transformers (ViTs) [5, 6, 23, 36, 38]: a state adapter in early layers reduces state confusion by amplifying low-level cues such as texture and color, an object adapter in middle layers learns shape and part structure, and a composition adapter in late layers integrates primitives into complete concepts. By exploiting this property and specializing transformer depths for different aspects of compositional reasoning, the model learns primitive-specific representations that transfer across compositional classes.

To improve the stability of compositional decision boundaries, we introduce *Primitive-Guided Stabilization* (PGS), which anchors each composition prototype to directions from the state and object heads when its primitives recur across tasks. Finally, to alleviate catastrophic forgetting arising from inter-task prototype misalignment, we propose *Adapter–Space Transport* (AST). In the composition-IL setting, each task has unique LoRA adapters, so features and prototypes learned at different times live in task-specific adapter spaces. These task-specific adapter spaces are not directly interoperable, limiting the use of cross-space evidence for previously learned compositions [49]. AST addresses this by fitting a lightweight affine relation from each past adapter space to the *current* one using only *current-task* image pairs (rehearsal-free), and transporting the anchored unit prototypes from past tasks into the current adapter geometry. Overall, our key contributions can be summarized as follows:

- We introduce **LSA-PAST**, a parameter-efficient composition-IL framework with novel depth-specialized adaptation and rehearsal-free prototype transport across task-specific adapter spaces.
- We curate a new composition-IL benchmark for evaluating novel state–object combinations and recurring primitives across tasks.
- We provide strong baselines, ablations, and analysis showing improved accuracy-forgetting trade-offs on composition-IL benchmarks.

2. Related Work

2.1. Class-Incremental Learning

Class-Incremental Learning (CIL) aims to learn new classes while retaining prior knowledge [56]. Classical methods such as EWC, LwF, and iCaRL respectively regularize parameter changes, distill from previous models, or combine exemplar replay with nearest-mean classification [20, 27, 39]. Exemplar-free CIL avoids storing past samples by regularizing the current model or allocating new capacity per task [15, 56], but still ignores compositional structure. Beyond the standard setting where tasks introduce disjoint class sets, related fields include multi-label CIL (MLCIL) [4, 41], which handles multi-label predictions per image, and blurry-CIL [1, 33], which allows overlapping class distributions across tasks. Related continual compositional works study continual compositional zero-shot learning (CCZSL) [31, 51] aimed at progressively improving zero-shot compositional generalization [28]. Composition-IL instead focuses on learning new state–object compositions, even when states and objects recur across tasks.

CIL with pre-trained models has shifted from training from scratch to adapting frozen pre-trained models (PTMs) [6] with parameter-efficient methods [45, 54]. PTMs encode rich visual knowledge and constrain plasticity to a low-dimensional subspace, reducing representation drift across tasks. We therefore adapt PTMs for composition-IL with parameter-efficient fine-tuning (PEFT), broadly categorized into prompt-based and adapter-based approaches.

Prompt-based methods introduce learnable tokens that steer a frozen backbone with high parameter efficiency [19, 24]. L2P retrieves prompts through query-key matching [45], Dual-Prompt separates shared and task-specific prompts [44], CODA-Prompt composes multiple prompts without full fine-tuning [40], and CompILer [52] extends this idea to composition-IL with multiple L2P-style prompt pools routed into state, object, and composition heads. Despite their efficiency, prompt-based methods often converge more slowly and are more difficult to optimize than adapter-based methods [3].

Adapter-based methods inject lightweight modules into transformer blocks to shape intermediate features, mitigating forgetting by localizing updates to specific subspaces [12]. Notable methods include EASE [55], which adds task-specific expandable modules, CL-LoRA [15], which employs a dual-adapter design, and TUNA [43], which combines entropy-guided task-specific adapters with a fused universal adapter. However, these methods typically treat classes as independent entities, ignoring the compositional structure of images. Our method addresses this gap through depth specialization and cross-subspace alignment to stabilize novel compositional pairs.

2.2. Hierarchical Processing in Vision Transformers

Recent studies reveal that ViTs [6] process visual information through distinct stages of integration across depth [5, 23, 36, 38]. *Early layers* (e.g., 1-4) primarily extract disentangled primitive attributes such as color, texture, and simple edges. These layers exhibit highly local-

ized receptive fields, resembling early convolutional filters that capture low-level statistics and fine-grained appearance cues [5, 38]. *Middle layers* (e.g., 5-8) progressively integrate these primitives into structured, object-level representations. As receptive fields expand, attention begins to aggregate features across parts, capturing relationships between components that define an object's shape and spatial layout [23, 36]. *Late layers* perform global operations over integrated representation, where attention heads attend globally across the entire image [38], capturing contextual relationships between objects and implementing operations that generalize to novel feature combinations [23, 36]. This progression motivates our depth-specialized design. Unlike uniform approaches [15], we align adapter placement within stages of the ViT layers based on this evidence.

3. Preliminaries

3.1. Problem Setup

In composition-IL, visual concepts of images are expressed as compositions of two primitive components: the *state* and an *object*. Let the state set be $\mathcal{S} = \{s_1, s_2, \ldots, s_n\}$ and the object set be $\mathcal{O} = \{o_1, o_2, \ldots, o_m\}$. Each composition class is defined as $c = \langle s, o \rangle$, where $s \in \mathcal{S}$ and $o \in \mathcal{O}$, forming the global set of possible compositions $\mathcal{C} \subseteq \mathcal{S} \times \mathcal{O}$. Learning proceeds over a sequence of N tasks $\mathcal{T} = \{\mathcal{T}_1, \mathcal{T}_2, \ldots, \mathcal{T}_N\}$, where each task $\mathcal{T}_t$ introduces a disjoint set of composition classes $\mathcal{C}^{(t)} \subset \mathcal{C}$, such that $\mathcal{C}^{(i)} \cap \mathcal{C}^{(j)} = \emptyset$ for any $i \neq j$. Unlike compositions, the primitive classes (states and objects) may recur across tasks, allowing knowledge of previously seen primitives to benefit new compositions. Each task $\mathcal{T}_t$ provides a labeled dataset $\mathcal{D}^{(t)} = \{(x_i, y_i)\}$, where each label $y_i = \langle s_i, o_i \rangle$ denotes the composition of state s_i and object o_i. Furthermore, we follow the challenging **rehearsal-free setting**, where no exemplars from previous tasks are leveraged for knowledge replay [9, 40, 55]. The model must learn to recognize the new compositions while maintaining performance on all previously seen compositions.

3.2. Adapter-Based Composition-IL

Following prior work [9, 15, 52, 55], we adopt a pretrained ViT [6] and perform adapter-based incremental learning with LoRA [18]. In this section, we introduce main components of our method in forming task-specific adapter spaces with LoRA and prototype-classifier for composition-IL.

Low-Rank Adaptation: LoRA [18] enables PEFT of pretrained models by freezing the backbone weights and injecting trainable low-rank adapters into the attention projections of transformer layers. For a pre-trained weight matrix $\mathbf{W_0} \in \mathbb{R}^{d\times k}$, LoRA modifies the feature extraction as:

$$\mathbf{y} = \mathbf{W_0}\mathbf{x} + \Delta\mathbf{W}\,\mathbf{x}, \tag{1}$$

where $\mathbf{x}$ and $\mathbf{y}$ denote the input and output activations, respectively, and $\Delta\mathbf{W} = \mathbf{UV}$ is a low-rank adaptation. Here, $\mathbf{V} \in \mathbb{R}^{r\times k}$ down-projects into a rank-r subspace and $\mathbf{U} \in \mathbb{R}^{d\times r}$ projects back to the output. Only $(\mathbf{U}, \mathbf{V})$ are trainable, adding $r(d+k)$ parameters while retaining the pretrained backbone. We apply LoRA to the Q/K/V projection matrices of the pre-trained ViT [6]. To reduce cross-task interference while preserving parameter efficiency, we instantiate *one LoRA adapter per task*: for task t we optimize only the rank-r updates $\{\Delta\mathbf{W}^{(t)}\}$ in selected layers, while all prior adapters $\{\Delta\mathbf{W}^{(t')}\}_{t'<t}$ remain frozen.

Prototype-Classifier for Composition-IL: Standard CIL methods [15, 43, 55] pair the backbone with a single linear head trained with cross-entropy on each task. In our depth-specialized setting for composition-IL, we instead use three heads during training (state, object, composition), but at the end of task t, only the composition head is replaced with a prototype classifier built from task-t features. Let $f^{(t)}_{\text{comp}}(x) \in \mathbb{R}^d$ be the composition feature from the final ViT layer within the task-t adapter space, and $\widehat{z}^{(t)}_{\text{comp}}(x) = \text{unit}\big(f^{(t)}_{\text{comp}}(x)\big)$ its L2-normalized version, where we define $\text{unit}(\mathbf{v}) = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}$. For each composition class $c \in \mathcal{C}^{(t)}$, we compute the empirical prototype as:

$$p^{\text{emp},(t)}_c = \text{unit}\Big(\frac{1}{|\mathcal{D}^{(t)}(c)|}\sum_{x_i\in\mathcal{D}^{(t)}(c)} \widehat{z}^{(t)}_{\text{comp}}(x_i)\Big), \tag{2}$$

where $\mathcal{D}^{(t)}(c) = \{x_i \in \mathcal{D}^{(t)} : y_i = c\}$. The resulting prototypes form the composition head used for inference, while the state and object heads remain standard linear classifiers for auxiliary supervision.

4. Methodology

4.1. Overview

LSA-PAST combines three components for composition-IL. First, LSA attaches task-specific LoRA adapters to different ViT depths for state, object, and composition reasoning. Second, PGS anchors empirical composition prototypes using recurring state and object directions. Third, AST maps old prototypes into new task adapter spaces through a rehearsal-free affine transform fitted only on current-task images. Together, depth-specialized LoRA and prototype-anchored transport balance *stability* from frozen adapters with *plasticity* for new compositions (Fig. 2).

4.2. Layer-Specific Adaptation with LoRA

We introduce **LSA**, with LoRA specialization modules applied at selected depths of a ViT. Motivated by evidence that ViT representations evolve with depth [5, 23, 36, 38], we allocate separate LoRA branches for states, objects, and compositions to different depth ranges instead of adapting all layers uniformly. This depth specialization exploits the ViT hierarchy and allocates adaptation capacity only

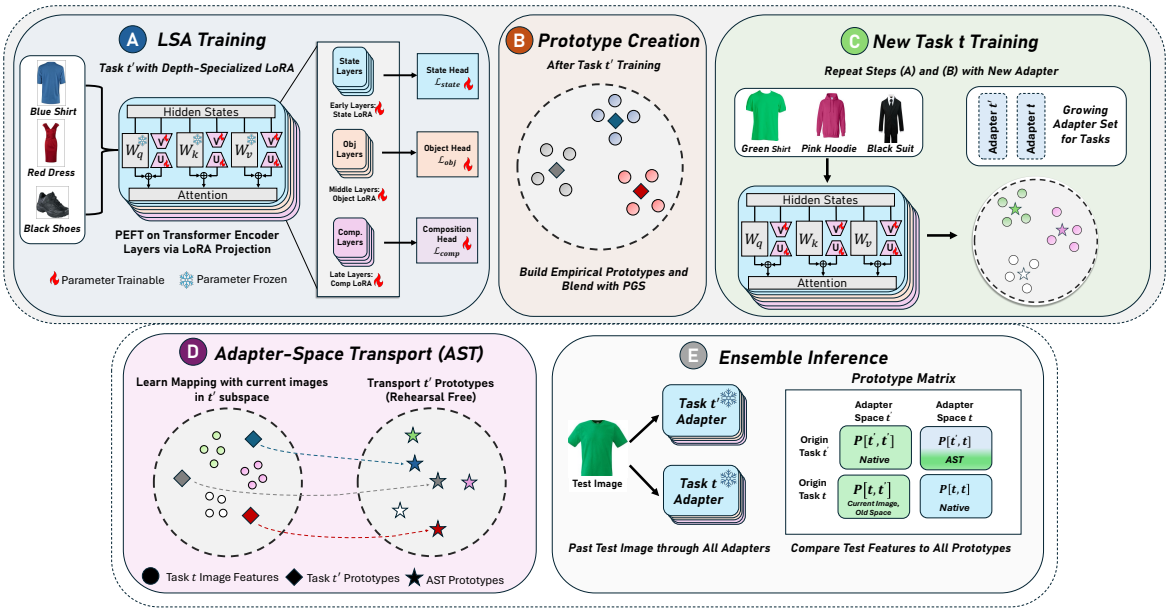


Figure 2. **(A)** We adapt a frozen ViT with depth-specialized LoRA feeding three heads. **(B)** Post-training, empirical prototypes are blended with primitive directions (PGS) and stored. **(C)** Subsequent tasks instantiate fresh adapters for new compositions. **(D)** Using current-task images as transient correspondences, AST fits an affine map $\phi_{t'\to t}$ to transport past prototypes into the current adapter space without rehearsal. **(E)** Inference ensembles cosine similarities across all adapter spaces, recognizing both new and previously learned compositions.

where needed, providing finer parameter control than uniform LoRA. In the incremental setting, we instantiate a ***separate set of adapters per task***; once a task finishes, its adapters are frozen and a fresh set is created for the subsequent task.

LoRA-Branch Configuration: Let $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$ denote a frozen attention projection matrix (query, key, or value) in the ViT backbone. For a given layer ℓ, we augment $\mathbf{W}_0$ with at most one low-rank adaptation, $\Delta\mathbf{W}$ (Eq.1). Rather than applying uniform LoRA across all layers, we exploit the hierarchical processing structure of ViTs. We partition the encoder into three disjoint depth ranges aligned with this hierarchy. This design helps mitigate state confusion by allocating early-layer adaptation to state cues such as texture and color, which are often suppressed by object-dominant representations in deeper layers. For layer ℓ assigned to branch $b \in \{\text{state}, \text{obj}, \text{comp}\}$, the depth-specialized adaptation becomes:

$$\Delta\mathbf{W}_\ell^b = \mathbf{U}_\ell^b \mathbf{V}_\ell^b, \tag{3}$$

where $\mathbf{U}_\ell^b \in \mathbb{R}^{d\times r_b}$ and $\mathbf{V}_\ell^b \in \mathbb{R}^{r_b\times k}$ are trainable with $r_b \ll \min(d,k)$. This configuration enables fine-grained control: each branch can use different ranks $(r_{\text{state}}, r_{\text{obj}}, r_{\text{comp}})$ while maintaining parameter efficiency through non-overlapping layer assignments. During training on task t, only the adapters for the current task $\{\mathbf{U}_\ell^{b,t}, \mathbf{V}_\ell^{b,t}\}$ are updated and all previous task adapters $\{\mathbf{U}_\ell^{b,t'}, \mathbf{V}_\ell^{b,t'}\}_{t'<t}$ remain frozen.

Multi-Branch Training Supervision: To leverage the depth-specialized LoRA adaptations, we introduce a multi-branch training scheme with dedicated classification heads for each semantic level. We maintain three linear classification heads that are updated incrementally across tasks, each parameterized by a weight matrix: $\mathbf{W}_{\text{state}} \in \mathbb{R}^{|\mathcal{S}|\times d}$, $\mathbf{W}_{\text{obj}} \in \mathbb{R}^{|\mathcal{O}|\times d}$, and $\mathbf{W}_{\text{comp}} \in \mathbb{R}^{|\mathcal{C}|\times d}$, where $\mathcal{S}$, $\mathcal{O}$, and $\mathcal{C}$ denote the sets of states, objects, and compositions, respectively. Each head is trained using cross entropy loss:

$$\mathcal{L}_{\text{comp}}^{(t)}(x,y) = -\log \frac{\exp\big([\mathbf{W}_{\text{comp}}\,\hat{z}_{\text{comp}}^{(t)}(x)]_y\big)}{\sum_{c\in\mathcal{C}^{(t)}} \exp\big([\mathbf{W}_{\text{comp}}\,\hat{z}_{\text{comp}}^{(t)}(x)]_c\big)} \tag{4}$$

where $[\cdot]_c$ denotes the c-th element of the logit vector. The losses $\mathcal{L}_{\text{state}}$ and $\mathcal{L}_{\text{obj}}$ are defined analogously. During training, the weighted multi-task loss backpropagates gradients through only the *current task's* adapters. At inference, linear composition head is replaced with prototype-based classification (Eq. 2), using prototypes maintained across tasks.

4.3. Prototype Anchoring with Primitive-Guided Stabilization

Composition-IL naturally recurs primitives across tasks. For example, after learning "Red Shoe" in Task 1, Task 2 might introduce "Red Hat" (recurring state) or "Blue Shoe" (recurring object). PGS leverages this recurrence by blending empirical prototypes with primitive-guided directions at the end of each task. This provides a recurrence-dependent stabilization mechanism while preserving the task-native empirical representation.

Blend prototypes with the final primitive direction: After training task t, for each composition class introduced during that task, we first build empirical prototypes by averaging unit-normalized features from the training samples per compositional class (and re-normalizing) to obtain $p_c^{\text{emp},(t)}$ (Eq. 2). We then construct the end-of-task PGS direction from the primitive head weights after training task t. Let $\mathbf{w}_s$ and $\mathbf{w}_o$ denote the weight vectors corresponding to state s and object o:

$$\mathbf{d}_{\text{PGS}}^{\text{end}}(c) = \text{unit}\left(\text{unit}(\mathbf{w}_s) + \text{unit}(\mathbf{w}_o)\right). \tag{5}$$

Although $\mathbf{w}_s$ and $\mathbf{w}_o$ arise from their respective depth-specific state and object representations, PGS uses their normalized classifier weights as discriminative primitive directions rather than empirical feature prototypes, consistent with prior work demonstrating that normalized classifier weights encode meaningful class directions in representation space [37]. PGS uses these directions as weak cross-depth primitive guidance rather than assuming identical feature geometry. To stabilize prototypes across tasks, we lightly blend $p_c^{\text{emp},(t)}$ with $\mathbf{d}_{\text{PGS}}^{\text{end}}(c)$ using a recurrence-weighted factor η_c to obtain the final native prototype in the task-t adapter space:

$$p_c^{(t)} \leftarrow \text{unit}\big((1-\eta_c)\, p_c^{\text{emp},(t)} + \eta_c\, \mathbf{d}_{\text{PGS}}^{\text{end}}(c)\big). \tag{6}$$

Blending is applied *once*, when the compositional class first appears and prototypes from earlier tasks are not re-blended thereafter. Furthermore, the blending weight η_c is determined by primitive recurrence at the time the composition was introduced. Let $\mathcal{S}^{<t}$ and $\mathcal{O}^{<t}$ denote states and objects seen before task t. For each composition $c = \langle s, o\rangle$, we set:

$$\eta_c = \begin{cases} \eta_{\text{both}}, & \text{if } s \in \mathcal{S}^{<t} \text{ and } o \in \mathcal{O}^{<t}, \\ \eta_{\text{one}}, & \text{if } (s \in \mathcal{S}^{<t}) \oplus (o \in \mathcal{O}^{<t}), \\ 0, & \text{if } s \notin \mathcal{S}^{<t} \text{ and } o \notin \mathcal{O}^{<t}. \end{cases} \tag{7}$$

When neither primitive has been seen ($\eta_c = 0$), $p_c^{(t)}$ remains equal to the empirical prototype $p_c^{\text{emp},(t)}$. When primitives recur, the state and object classifiers retain and refine their weight vectors across tasks, providing reusable discriminative guidance for blending. Therefore, PGS anchors new composition prototypes using learned primitive directions to improve stability across tasks.

4.4. Adapter-Space Transport

Each task $\mathcal{T}_t$ instantiates its own LoRA adapters (Sec. 4.2), which are frozen once training completes. Consequently, composition embeddings from different tasks live in *adapter-specific* feature spaces. We refer to the feature space induced by the adapters trained for task a as *adapter space* a. At the end of task t, we maintain a block-structured prototype matrix P, where each block $P[t', a] \in \mathbb{R}^{|\mathcal{C}^{(t')}| \times d}$ stores unit prototypes for classes introduced at task t' but expressed in adapter space $a \le t$. Diagonal blocks contain native PGS prototypes, while off-diagonal blocks store cross-subspace prototypes.

For a new task t, we first fill $P[t, t]$ with PGS-anchored prototypes from current-task images under the task-t adapters. For the same classes, we obtain $P[t, a]$ for each earlier adapter space $a < t$ by passing current-task images through frozen past adapters. In contrast, for classes introduced in earlier tasks $t' < t$, we lack their images in the rehearsal-free setting [14, 15, 55] and cannot directly compute their prototypes under the current adapters. Their native prototypes remain reliable in their originating adapter spaces, but native-space scoring leaves the task-specific spaces isolated. AST instead makes these spaces interoperable by using current-task images as transient correspondences to express prior classes in later adapter spaces without rehearsal.

Cross-Subspace Alignment: For a current task t and any previous task index $t' < t$, we take the same current-task image $x \in \mathcal{D}^{(t)}$ and extract L2-normalized composition embeddings under both adapter sets: $\hat{z}_{\text{comp}}^{(t')}(x) \in \mathbb{R}^d$ from the frozen task-t' adapters and $\hat{z}_{\text{comp}}^{(t)}(x) \in \mathbb{R}^d$ from the current adapters. The pairs $\big(\hat{z}_{\text{comp}}^{(t')}(x), \hat{z}_{\text{comp}}^{(t)}(x)\big)$ provide transient correspondences that identify a cross-subspace relation between adapter spaces [9, 11, 17, 47, 49]. Using only these current-task correspondences, we fit a regularized affine map directly from each past adapter space t' to the current space t and apply it to prior prototypes without rehearsal. By default, we use all current-task images and vary this number in our transport-budget ablation.

We define the adapter-space transport map $\phi_{t'\to t} : \mathbb{R}^d \to \mathbb{R}^d$ as an affine transformation:

$$\phi_{t'\to t}(u) \;=\; \mathbf{A}_{t'\to t}\, u \;+\; \mathbf{b}_{t'\to t}, \tag{8}$$

where $\mathbf{A}_{t'\to t} \in \mathbb{R}^{d\times d}$ and $\mathbf{b}_{t'\to t} \in \mathbb{R}^d$. For a fixed past adapter index $t' < t$, we estimate $(\mathbf{A}, \mathbf{b}) \equiv (\mathbf{A}_{t'\to t}, \mathbf{b}_{t'\to t})$ at the end of task t by minimizing:

$$\min_{\mathbf{A},\mathbf{b}} \sum_{x_i \in \mathcal{D}^{(t)}} \big\| \hat{z}_{\text{comp}}^{(t)}(x_i) - \big(\mathbf{A}\, \hat{z}_{\text{comp}}^{(t')}(x_i) + \mathbf{b}\big) \big\|_2^2 \;+\; \lambda \|\mathbf{A}\|_F^2, \tag{9}$$

where the ridge term $\lambda\|\mathbf{A}\|_F^2$ regularizes the linear component while leaving the bias unconstrained. With unit-normalized features, minimizing this objective aligns old-adapter directions to current-adapter directions on the current-task distribution. We fit $\phi_{t'\to t}$ once per pair (t', t) at the end of task t only into the current adapter space and *do not refit maps between earlier adapters*.

Prototype Transport: After fitting maps for all $t' < t$, we transport all previously seen prototypes into the current adapter space. At the end of task t, we already have

the native prototype $p_c^{(t)}$ for each new class $c \in \mathcal{C}^{(t)}$ from PGS (Eq. 6), stored as the native diagonal block $P[t,t]$ of the prototype matrix, and we have directly computed cross-subspace prototypes $P[t,a]$ for all $a < t$ by passing current-task images through older adapters. More generally, for any composition class c introduced at task t_c, its stored, blended prototype $p_c^{(t_c)}$ remains in its originating adapter space as the diagonal block $P[t_c, t_c]$.

Applying $\phi_{t' \to t}$ to these native prototypes yields a transported view in the current adapter space (adapter space t):

$$\tilde{p}_c^{(t' \to t)} \;=\; \mathrm{unit}\big(\mathbf{A}_{t' \to t}\, p_c^{(t')} + \mathbf{b}_{t' \to t}\big), \qquad (10)$$

for all classes c whose origin task is $t' = t_c < t$. The transported prototypes $\tilde{p}_c^{(t' \to t)}$ are stored in the above-diagonal block $P[t', t]$. Intuitively, $\phi_{t' \to t}$ determines how old classes would be embedded by the current adapters, based on how current-task images move between adapter spaces. This procedure is performed after every task: at the end of task t we build new native prototypes $P[t,t]$ and their cross-subspace views $P[t,a]$ for all $a < t$ using current images, and fit a map $\phi_{t' \to t}$ for each $t' < t$ and use it to update the column $P[t', t]$. No past-task images are stored or replayed, the only persisted state comprises frozen adapters, PGS-anchored prototypes, and their transported versions.

4.5. Optimization Objective

Training. We train with a simple weighted multi-task objective over the three linear classification heads,

$$\mathcal{L} \;=\; \mathcal{L}_{\text{comp}} \;+\; \lambda_{\text{prim}}\big(\mathcal{L}_{\text{state}} + \mathcal{L}_{\text{obj}}\big), \qquad (11)$$

where each loss term $\mathcal{L}_{\text{comp}}$, $\mathcal{L}_{\text{state}}$, and $\mathcal{L}_{\text{obj}}$ is computed as standard cross-entropy for its respective classification head and λ_{prim} is the primitive weight for the state and object losses. Moreover, PAST does not alter the loss function, adding no extra forward-time cost during training.

Inference. At task t inference, we discard the linear composition head and classify with cosine similarity to stored prototypes using ensemble over adapter spaces. For each adapter space $a \le t$, the composition branch produces a feature $f_{\text{comp}}^{(a)}(x)$ for an image x, which we unit-normalize to obtain $\hat{z}_{\text{comp}}^{(a)}(x)$. Each composition class $c \in \mathcal{C}^{\le t}$ is first introduced at task t_c and has a native prototype $p_c^{(t_c)}$ in adapter space t_c. Across subsequent tasks, we populate a prototype matrix that stores cross-subspace views of each class. When a prototype for class c is available in adapter space a, we denote it by $p_c^{(a)}$. We predict the label for x by ensembling cosine similarities across all adapter spaces up to t:

$$\widehat{y} \;=\; \arg\max_{c \in \mathcal{C}^{\le t}} \sum_{a=1}^{t} w_{c,a}\, \hat{z}_{\text{comp}}^{(a)}(x)^{\top} p_c^{(a)}, \qquad (12)$$

We assign full weight to the native adapter of class c ($w_{c,t_c} = 1$) and a smaller constant weight to all other available adapter spaces ($w_{c,a} = 0.1$ for $a \neq t_c$ in all experiments).

5. Experiments and Results

5.1. Experimental Setup

Datasets. We evaluate on three composition-IL benchmarks: Split-Clothing [50], Split-UT-Zappos [48], and our proposed Split-C-GQA dataset derived from C-GQA [34]. Existing work evaluates only on smaller-scale, heavily recurring benchmarks while our Split-C-GQA introduces a larger, less-redundant setting. Split-Clothing has 35 compositions (9 states, 8 objects) across 5 tasks with all primitives recurring, Split-UT-Zappos has 80 compositions (15 states, 12 objects) with 9 states and all objects recurring, Split-C-GQA has 100 compositions (31 states, 31 objects) with 13 states and 18 objects recurring. Detailed statistics are summarized in the supplemental material.

Baselines and Metrics. We opt to compare with Classic CIL [20, 27, 39], prompt-based methods [40, 44, 45, 52], and adapter-based CIL [15, 43, 55], using LAMDA-PILOT [42, 54, 56] and PyCIL [53] implementations with tested optimal hyperparameters. iCaRL is the only method which uses exemplars. Results come from *our re-implementations* using the same set of 3 random seeds.

Metrics. The metrics reported are Average Accuracy (**Avg. Acc**) over all composition classes seen up to the current task, computed by evaluating the model on every previously seen task after each training step and averaging their accuracies and Forgetting (**FTT**), which quantifies the degradation on past tasks by comparing each task's current accuracy to its best accuracy observed earlier.

Implementation Details. We leverage ViT-B/16 [6] with ImageNet-21k pretrain following all prior works [15, 52, 55]. The multitask loss is set at $\lambda_{\text{prim}} = 0.1$. We set $(\eta_{\text{both}}, \eta_{\text{one}}) = (0.2, 0.1)$ for Split-Clothing and Split-UT-Zappos, and $(0.15, 0.075)$ for Split-C-GQA. All hyperparameters are selected once using only the first-task split and kept fixed for all subsequent tasks; we do not perform per-task hyperparameter tuning. A detailed complexity analysis, including storage, inference FLOPs, latency, and per-task training time, is provided in the supplemental material.

5.2. Experimental Results

Table 1 reports Avg Acc and FTT across all benchmarks, including a longer 10-task Split-UT-Zappos setting. LSA-PAST achieves the strongest overall accuracy-forgetting trade-off with low parameter overhead by combining depth-specialized adapters, PGS-anchored prototypes, and AST-based cross-task alignment. On Split-Clothing, LSA-PAST reaches 91.86% Avg Acc with only 2.61% FTT, improving over CompILer (86.36%, 7.76%) while using fewer trainable parameters. On Split-C-GQA, we also obtain the best

Method type	Method	%Params	Split-Clothing		Split-UT-Zappos				Split-C-GQA	
			5 tasks		5 tasks		10 tasks		5 tasks	
			Avg Acc % (↑)	FTT % (↓)	Avg Acc % (↑)	FTT % (↓)	Avg Acc % (↑)	FTT % (↓)	Avg Acc % (↑)	FTT % (↓)
Classic CIL	EWC [20]	100	47.89±0.87	52.75±0.44	37.59±2.06	55.70±2.76	24.63±0.94	61.31±2.29	38.69±1.39	38.98±1.53
	LwF [27]	100	49.96±0.68	44.22±0.53	40.15±0.43	49.61±0.68	30.38±1.41	58.15±0.20	45.95±0.72	23.21±1.05
	iCaRL [39]	100	68.65±0.41	31.74±1.89	37.78±2.14	55.06±3.50	31.40±1.96	59.65±2.40	33.40±0.40	53.32±0.90
Prompt-based	L2P [45]	0.2	83.16±0.50	10.97±2.02	35.26±0.81	28.17±0.05	31.65±0.16	31.02±1.62	39.39±0.24	27.27±1.67
	Dual-Prompt [44]	0.3	85.47±0.39	9.47±1.26	36.59±2.01	**18.90**±1.63	33.01±1.65	24.61±1.11	39.52±0.33	17.13±0.94
	CODA-Prompt [40]	4.3	83.07±2.56	11.04±1.95	38.92±0.27	20.55±2.87	31.40±0.36	30.54±2.63	42.79±0.51	18.63±1.16
	Sim-CompILer [52]	3.1	84.68±0.58	9.98±0.34	42.27±0.63	23.89±0.61	33.30±0.10	30.31±0.03	41.08±0.27	21.20±0.18
	CompILer [52]	3.1	86.36±0.36	7.76±0.68	**44.17**±1.41	23.28±2.00	34.43±0.07	28.69±0.82	42.49±0.35	20.36±0.10
Adapter-based	EASE [55]	2.2	76.77±0.18	9.79±0.18	39.58±0.14	24.26±0.33	34.65±0.22	**19.60**±0.43	**48.19**±0.15	18.79±0.13
	TUNA [43]	2.2	**89.53**±0.67	**4.59**±1.22	43.24±1.38	24.09±0.89	**40.90**±0.45	23.23±0.43	45.08±0.60	23.32±0.24
	CL-LoRA [15]	0.5	83.07±0.15	6.70±0.51	43.00±1.44	28.02±1.71	37.71±0.80	29.25±1.45	36.89±0.09	**13.58**±1.06
	LSA-PAST (ours)	1.8	**91.86**±0.99	**2.61**±0.32	**44.71**±0.68	**14.22**±2.18	**39.78**±0.86	**16.78**±0.57	**48.57**±1.09	**11.58**±0.46

Table 1. Average accuracy and forgetting on Split-Clothing, Split-UT-Zappos under 5-task and 10-task, and Split-C-GQA. %Params denotes cumulative additional parameters relative to the ViT-B/16 backbone. **Best** and **Second-Best** results highlighted.

Type	Method	Split-Clothing		Split-C-GQA	
		State %	Object %	State %	Object %
Classic	EWC [20]	86.49±0.97	52.72±1.30	52.91±0.28	60.42±1.91
	LwF [27]	87.11±0.66	54.57±0.69	**58.33**±0.42	66.32±0.31
	iCaRL [39]	91.21±1.05	71.70±0.99	46.96±0.27	55.61±0.45
Prompt	L2P [45]	85.41±0.23	95.42±0.42	51.65±1.54	63.53±1.04
	Dual-Prompt [44]	86.89±0.37	95.87±0.53	49.52±0.69	64.15±0.37
	Sim-CompILer [52]	87.14±0.98	95.91±0.12	51.85±0.45	63.89±0.38
	CompILer [52]	87.76±1.29	**96.46**±0.08	52.70±0.20	64.54±0.22
Adapt	EASE [55]	77.50±0.16	93.73±0.14	58.19±0.14	**69.93**±0.17
	TUNA [43]	**92.73**±0.93	95.84±0.19	56.38±0.23	65.66±0.09
	CL-LoRA [15]	84.81±0.27	96.00±0.11	45.26±0.17	59.28±0.38
	Ours	**94.24**±0.21	**96.64**±0.25	**58.74**±0.28	**71.01**±0.23

Table 2. State and Object Accuracy (%) on Split-Clothing and Split-C-GQA (5 tasks). **Best** and **Second-Best** results highlighted.

Avg Acc and lowest FTT, showing that the method remains effective on a larger benchmark with lower primitive recurrence. On 5-task Split-UT-Zappos, LSA-PAST slightly improves Avg Acc over CompILer (44.71% vs. 44.17%) but substantially reduces forgetting (14.22% vs. 23.28%), demonstrating superior stability. Under the longer 10-task schedule, TUNA obtains higher Avg Acc, but LSA-PAST achieves lower FTT (16.78% vs. 23.23%), demonstrating that our method better preserves prior compositions as task sequences grow. Overall, LSA-PAST delivers the most consistent accuracy-forgetting balance across all settings.

Table 2 reports state and object accuracy on Split-Clothing and Split-C-GQA. LSA-PAST achieves the best primitive-level accuracy across both benchmarks, supporting depth-specialized LoRA for preserving state/object knowledge and reducing state confusion while incrementally learning compositions.

5.3. Ablation Study

Component and Transport Ablation. We ablate the components of LSA-PAST in Table 3. The unaligned ensemble performs poorly, confirming that scores cannot be reliably aggregated across task-specific adapter spaces without alignment. Native-space inference provides a strong transport-free baseline by evaluating each class only in its originating space. Adding PGS consistently reduces forgetting in both Native and AST inference, supporting its role in stabilizing composition prototypes through recurring primitive knowledge. With PGS fixed, AST improves over Native by retaining reliable native evidence while incorporating lower-weight aligned evidence from non-native spaces, increasing Avg. Acc. from 90.74% to 91.86% on Split-Clothing and from 41.67% to 44.71% on Split-UT-Zappos. Finally, under the same LSA+PGS backbone and ensemble protocol, AST outperforms previous drift compensation methods LDC [11] and DPCR [17], yielding the strongest overall accuracy-forgetting trade-off.

LSA	PGS	Inference	Split-Clothing		Split-UT-Zappos	
			Avg Acc % (↑)	FTT % (↓)	Avg Acc % (↑)	FTT % (↓)
✓	✓	Unaligned Ens.	75.71±0.39	28.32±0.53	29.92±0.37	53.21±1.51
✓	✗	Native	90.29±0.65	4.97±0.48	40.85±0.93	17.56±1.71
✓	✓	Native	90.74±0.65	3.91±0.37	41.67±0.57	14.81±1.85
✓	✗	**AST Ens. (ours)**	90.81±0.27	4.89±0.87	44.58±1.56	18.63±3.17
✓	✓	LDC Ens. [11]	91.41±0.40	3.49±0.73	43.73±0.77	15.09±1.74
✓	✓	DPCR Ens. [17]	91.25±0.84	3.72±0.58	44.07±0.74	15.26±1.98
✓	✓	**AST Ens. (ours)**	**91.86**±0.99	**2.61**±0.32	**44.71**±0.68	**14.22**±2.18

Table 3. Component and inference ablation. Native scores each class only in its originating adapter space, while ensemble (Ens.) variants additionally aggregate available cross-space evidence.

PGS Qualitative Analysis. Fig. 3 shows representative PGS success and failure cases on Split-UT-Zappos and Split-C-GQA. PGS substantially reduces forgetting for visually distinctive states such as "Leather" and "Parked," where primitive information provides a reliable direction for prototype stabilization. In contrast, ambiguous states such as "Small" or "Synthetic" can produce slight degradation, suggesting that primitive anchoring is less reliable when state evidence is weak or visually subtle.

Oracle Prototype Alignment. For each past class, we construct an *oracle prototype* from held-out test images passed through the current adapter and compare it with the Uncorrected prototype from task t', LDC-corrected [11], DPCR-



Figure 3. PGS success and failure cases on Split-UT-Zappos and Split-C-GQA. Boxes show Δ forgetting, with green indicating reduced forgetting and red indicating no improvement.

	Split-Clothing				Split-C-GQA			
Task Gap	Uncorrected	LDC [11]	DPCR [17]	AST	Uncorrected	LDC [11]	DPCR [17]	AST
1	0.740	**0.898**	0.863	0.880	0.784	0.942	**0.945**	0.941
2	0.750	0.873	0.830	**0.897**	0.786	0.928	0.934	**0.945**
3	0.704	0.844	0.812	**0.886**	0.798	0.910	0.920	**0.948**
4	0.662	0.819	0.799	**0.905**	0.774	0.881	0.894	**0.952**
Overall	0.728	0.872	0.836	**0.889**	0.786	0.925	0.932	**0.944**

Table 4. Cosine similarity to oracle current-space prototypes.

corrected [17], and AST-transported prototypes using cosine similarity (Table 4). Results are grouped by task gap $t - t'$, while Overall averages all past-to-current task-pair comparisons. LDC and DPCR are slightly stronger at gap 1 on Split-Clothing and Split-C-GQA, respectively, but AST performs best at gaps 2-4 and achieves the highest overall alignment on both datasets. This demonstrates that AST more closely approximates the current-space location of old-class prototypes as task separation increases.

Effect of Number of Transported Pairs. The top panel of Fig. 4 varies the number of current-task feature correspondences used to fit each AST map. Accuracy improves rapidly with a small transport budget, while additional correspondences further reduce forgetting by refining cross-space alignment. Increasing the budget affects only post-task AST fitting and does not increase inference-time cost. The 95% markers show that near-maximum accuracy is reached with fewer correspondences than near-optimal forgetting, suggesting that coarse alignment is sufficient for accuracy gains, whereas stronger retention benefits from finer transport estimates.

Effect of Layer Assignment. The bottom panel of Fig. 4 compares depth-specialized LSA with alternative layer assignments, including uniform LoRA under a matched parameter budget, where identical adapters are attached to all layers for S/O/C. Our LSA assignment follows the *approximate* hierarchy observed in prior work [5, 23, 36]: state cues are assigned to early layers, object structure to middle layers, and global operation to the final layers. While the uniform design is competitive, LSA consistently improves Avg Acc and further reduces FTT using the same number of trainable parameters. Depth specialization aligns adaptation with this hierarchy, avoiding uniform updates that can *entangle* unrelated features. This design also enables depth-wise control (e.g., different LoRA ranks per depth), offering a more flexible alternative to uniform LoRA.

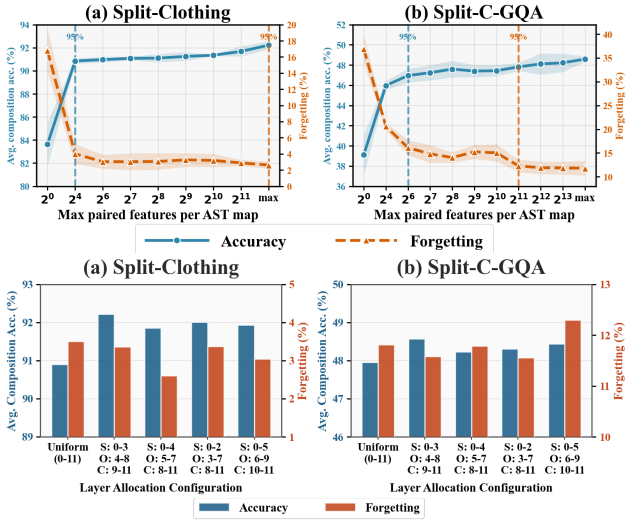


Figure 4. Top: effect of AST transport-pair budget on accuracy and forgetting. Bottom: LSA ablation with explicit state (S), object (O), and composition (C) layer ranges.

6. Conclusion

In this paper, we propose LSA-PAST, a framework for composition-IL that exploits the hierarchical processing structure of vision transformers and a novel, rehearsal-free cross-subspace alignment method for aligning old prototypes with current task subspaces. Through extensive experiments across multiple benchmark datasets, we demonstrate that our method achieves SOA performance in accuracy-forgetting trade off. Comprehensive studies validate the importance of both depth-specialized adapter placement and AST to overall performance. Finally, we provide contributions to an emerging field of incremental learning, introducing a framework that lays foundational groundwork for future research in compositional continual models.

References

[1] Jihwan Bang, Heesu Kim, YoungJoon Yoo, Jung-Woo Ha, and Jonghyun Choi. Rainbow memory: Continual learning with a memory of diverse samples. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8218–8227, 2021. 2

[2] Fadi Boutros, Naser Damer, Florian Kirchbuchner, and Arjan Kuijper. Elasticface: Elastic margin loss for deep face recognition. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1578–1587, 2022. 1

[3] Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, et al. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence*, 5(3):220–235, 2023. 2

[4] Songlin Dong, Haoyu Luo, Yuhang He, Xing Wei, Jie Cheng, and Yihong Gong. Knowledge restore and transfer for multi-label class-incremental learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 18711–18720, 2023. 2

[5] Teresa Dorszewski, Lenka Tětková, Robert Jenssen, Lars Kai Hansen, and Kristoffer Knutsen Wickstrøm. From colors to classes: Emergence of concepts in vision transformers. In *World Conference on Explainable Artificial Intelligence*, pages 28–47. Springer, 2025. 2, 3, 8

[6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations*, 2021. 2, 3, 6

[7] Steven M Frankland and Joshua D Greene. Concepts and compositionality: in search of the brain's language of thought. *Annual review of psychology*, 71(1):273–303, 2020. 1

[8] Robert M French. Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences*, 3(4):128–135, 1999. 1

[9] Takuma Fukuda, Hiroshi Kera, and Kazuhiko Kawamoto. Adapter merging with centroid prototype mapping for scalable class-incremental learning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 4884–4893, 2025. 3, 5

[10] Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A Wichmann, and Wieland Brendel. Imagenet-trained cnns are biased towards texture; increasing shape bias improves accuracy and robustness. In *International conference on learning representations*, 2018. 1, 2

[11] Alex Gomez-Villa, Dipam Goswami, Kai Wang, Andrew D Bagdanov, Bartlomiej Twardowski, and Joost van de Weijer. Exemplar-free continual representation learning via learnable drift compensation. In *European Conference on Computer Vision*, pages 473–490. Springer, 2024. 5, 7, 8

[12] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608*, 2024. 2

[13] Shaozhe Hao, Kai Han, and Kwan-Yee K Wong. Learning attention as disentangler for compositional zero-shot learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15315–15324, 2023. 1

[14] Jiangpeng He and Fengqing Zhu. Exemplar-free online continual learning. In *2022 IEEE International Conference on Image Processing (ICIP)*, pages 541–545. IEEE, 2022. 5

[15] Jiangpeng He, Zhihao Duan, and Fengqing Zhu. Cl-lora: Continual low-rank adaptation for rehearsal-free class-incremental learning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 30534–30544, 2025. 1, 2, 3, 5, 6, 7

[16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 1

[17] Run He, Di Fang, Yicheng Xu, Yawen Cui, Ming Li, Cen Chen, Ziqian Zeng, and Huiping Zhuang. Semantic shift estimation via dual-projection and classifier reconstruction for exemplar-free class-incremental learning. In *Forty-second International Conference on Machine Learning*, 2025. 5, 7, 8

[18] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022. 2, 3

[19] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. In *European conference on computer vision*, pages 709–727. Springer, 2022. 2

[20] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017. 1, 2, 6, 7

[21] Brenden M Lake. *Towards more human-like concept learning in machines: Compositionality, causality, and learning-to-learn*. PhD thesis, Massachusetts Institute of Technology, 2014. 1

[22] Brenden M Lake, Tomer D Ullman, Joshua B Tenenbaum, and Samuel J Gershman. Building machines that learn and think like people. *Behavioral and brain sciences*, 40:e253, 2017. 1

[23] Michael Lepori, Alexa Tartaglini, Wai Keen Vong, Thomas Serre, Brenden M Lake, and Ellie Pavlick. Beyond the doors of perception: Vision transformers represent relations between objects. *Advances in Neural Information Processing Systems*, 37:131503–131544, 2024. 2, 3, 8

[24] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021. 2

[25] Jiashuo Li, Shaokun Wang, Bo Qian, Yuhang He, Xing Wei, Qiang Wang, and Yihong Gong. Dynamic integration of

task-specific adapters for class incremental learning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 30545–30555, 2025. 1

[26] Yanghao Li, Chao-Yuan Wu, Haoqi Fan, Karttikeya Mangalam, Bo Xiong, Jitendra Malik, and Christoph Feichtenhofer. Mvitv2: Improved multiscale vision transformers for classification and detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4804–4814, 2022. 1

[27] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2935–2947, 2017. 1, 2, 6, 7

[28] Zhen Li, Yuwei Wu, Chenchen Jing, Che Sun, Chuanhao Li, and Yunde Jia. Composition-incremental learning for compositional generalization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 6735–6743, 2026. 2

[29] Wenzhuo Liu, Xin-Jian Wu, Fei Zhu, Ming-Ming Yu, Chuang Wang, and Cheng-Lin Liu. Class incremental learning with self-supervised pre-training and prototype learning. *Pattern Recognition*, 157:110943, 2025. 1

[30] Xiaocheng Lu, Song Guo, Ziming Liu, and Jingcai Guo. Decomposed soft prompt guided fusion enhancing for compositional zero-shot learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 23560–23569, 2023. 1

[31] Sauda Maryam, Sara Nadeem, Faisal Qureshi, and Mohsen Ali. Prompt-based continual compositional zero-shot learning. *arXiv preprint arXiv:2512.09172*, 2025. 2

[32] Zichong Meng, Jie Zhang, Changdi Yang, Zheng Zhan, Pu Zhao, and Yanzhi Wang. Diffclass: Diffusion-based class incremental learning. In *European Conference on Computer Vision*, pages 142–159. Springer, 2024. 1

[33] Jun-Yeong Moon, Keon-Hee Park, Jung Uk Kim, and Gyeong-Moon Park. Online class incremental learning on stochastic blurry task boundary via mask and visual prompt tuning. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 11731–11741, 2023. 2

[34] Muhammad Ferjad Naeem, Yongqin Xian, Federico Tombari, and Zeynep Akata. Learning graph embeddings for compositional zero-shot learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 953–962, 2021. 6

[35] Muhammad Muzammal Naseer, Kanchana Ranasinghe, Salman H Khan, Munawar Hayat, Fahad Shahbaz Khan, and Ming-Hsuan Yang. Intriguing properties of vision transformers. *Advances in Neural Information Processing Systems*, 34: 23296–23308, 2021. 1

[36] Xu Pan, Aaron Philip, Ziqian Xie, and Odelia Schwartz. Dissecting query-key interaction in vision transformers. In *Advances in Neural Information Processing Systems*, pages 54595–54631, 2024. 2, 3, 8

[37] Hang Qi, Matthew Brown, and David G Lowe. Low-shot learning with imprinted weights. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5822–5830, 2018. 5

[38] Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34:12116–12128, 2021. 2, 3

[39] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 2001–2010, 2017. 1, 2, 6, 7

[40] James Seale Smith, Leonid Karlinsky, Vyshnavi Gutta, Paola Cascante-Bonilla, Donghyun Kim, Assaf Arbelle, Rameswar Panda, Rogerio Feris, and Zsolt Kira. Coda-prompt: Continual decomposed attention-based prompting for rehearsal-free continual learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11909–11919, 2023. 1, 2, 3, 6, 7

[41] Xiang Song, Kuang Shu, Songlin Dong, Jie Cheng, Xing Wei, and Yihong Gong. Overcoming catastrophic forgetting for multi-label class-incremental learning. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 2389–2398, 2024. 2

[42] Hai-Long Sun, Da-Wei Zhou, De-Chuan Zhan, and Han-Jia Ye. Pilot: A pre-trained model-based continual learning toolbox. *SCIENCE CHINA Information Sciences*, 68(4):147101, 2025. 6

[43] Yan Wang, Da-Wei Zhou, and Han-Jia Ye. Integrating task-specific and universal adapters for pre-trained model-based class-incremental learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 806–816, 2025. 2, 3, 6, 7

[44] Zifeng Wang, Zizhao Zhang, Sayna Ebrahimi, Ruoxi Sun, Han Zhang, Chen-Yu Lee, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, et al. Dualprompt: Complementary prompting for rehearsal-free continual learning. In *European conference on computer vision*, pages 631–648. Springer, 2022. 1, 2, 6, 7

[45] Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, and Tomas Pfister. Learning to prompt for continual learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 139–149, 2022. 1, 2, 6, 7

[46] Kun Wei, Muli Yang, Hao Wang, Cheng Deng, and Xianglong Liu. Adversarial fine-grained composition learning for unseen attribute-object recognition. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3741–3749, 2019. 2

[47] Fangwen Wu, Lechao Cheng, Shengeng Tang, Xiaofeng Zhu, Chaowei Fang, Dingwen Zhang, and Meng Wang. Navigating semantic drift in task-agnostic class-incremental learning. In *Forty-second International Conference on Machine Learning*, 2025. 5

[48] Aron Yu and Kristen Grauman. Fine-grained visual comparisons with local learning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 192–199, 2014. 6

[49] Lu Yu, Bartlomiej Twardowski, Xialei Liu, Luis Herranz, Kai Wang, Yongmei Cheng, Shangling Jui, and Joost van de Weijer. Semantic drift compensation for class-incremental

learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6982–6991, 2020. 2, 5

[50] Tian Zhang, Kongming Liang, Ruoyi Du, Xian Sun, Zhanyu Ma, and Jun Guo. Learning invariant visual representations for compositional zero-shot learning. In *European conference on computer vision*, pages 339–355. Springer, 2022. 6

[51] Yang Zhang, Songhe Feng, and Jiazheng Yuan. Continual compositional zero-shot learning. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, 2024. 2

[52] Yanyi Zhang, Binglin Qiu, Qi Jia, Yu Liu, and Ran He. Not just object, but state: compositional incremental learning without forgetting. *Advances in Neural Information Processing Systems*, 37:123182–123206, 2024. 1, 2, 3, 6, 7

[53] Da-Wei Zhou, Fu-Yun Wang, Han-Jia Ye, and De-Chuan Zhan. Pycil: a python toolbox for class-incremental learning. *SCIENCE CHINA Information Sciences*, 66(9):197101, 2023. 6

[54] Da-Wei Zhou, Hai-Long Sun, Jingyi Ning, Han-Jia Ye, and De-Chuan Zhan. Continual learning with pre-trained models: A survey. In *IJCAI*, pages 8363–8371, 2024. 2, 6

[55] Da-Wei Zhou, Hai-Long Sun, Han-Jia Ye, and De-Chuan Zhan. Expandable subspace ensemble for pre-trained model-based class-incremental learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 23554–23564, 2024. 1, 2, 3, 5, 6, 7

[56] Da-Wei Zhou, Qi-Wei Wang, Zhi-Hong Qi, Han-Jia Ye, De-Chuan Zhan, and Ziwei Liu. Class-incremental learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. 2, 6